AZURE DEVOPS GITHUB ACTIONS

CI/CD with YAML

A practical learning guide: YAML syntax, pipeline schemas, and build and deploy pipelines for .NET, Java, AKS and ACA.

```yaml
# the smallest useful pipeline
name: CI
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test
```

Part 1 YAML basics | Part 2 Schema tables | Part 3 Node.js pipeline
Part 4 .NET, Java, ACR to AKS, ACR to ACA | Part 5 Cheat sheet and next steps

How to use this guide

This guide collects everything from the learning session in one place. It starts with plain YAML, then explains the two pipeline schemas, then walks through real pipelines line by line. Read it top to bottom once, then use Part 5 as a quick reference.

Part	What you will learn	Why it matters
1. YAML basics	Maps, lists, strings, multi-line text, anchors, common gotchas	Every CI/CD tool describes pipelines in YAML. One wrong indent breaks the file.
2. Schema tables	Every key of Azure DevOps and GitHub Actions at each level, side by side	Lets you read any pipeline and know what each key controls.
3. Node.js pipeline	Build, test, publish, deploy in both tools, explained line by line	The simplest complete CI/CD flow. Everything else is a variation.
4. Azure scenarios	.NET and Java to App Service, Docker to ACR then AKS, Docker to ACR then ACA	The four most common Azure delivery patterns.
5. Cheat sheet	Key differences, AKS vs ACA, improvements to learn next	A map of what to study after this.

Colour legend for code samples

All YAML in this guide is syntax-highlighted. Each colour has a fixed meaning:

`key`	Keys (names of settings)	`- dash`	List item marker
`string`	Text values	`42`	Numbers
`true`	Booleans, block markers	`# note`	Comments
`$(x)`	Variable / expression	`task@1`	Task or action @ version

> **Before you start**
> Names such as `myacr`, `my-rg`, `my-dotnet-app` are placeholders. Replace them with your own resources.
> Action and task versions change over time. Check each action or task page for the latest version before using it.

1 YAML basics

The language every pipeline is written in

YAML is a human-readable format for configuration. CI/CD tools use it to describe pipelines as code, so the pipeline lives in your repository, is versioned, and is reviewed like any other file.

> **Core rules**
> **Indentation defines structure.** Use spaces, never tabs. Two spaces per level is the convention.
> Keys and values are **case-sensitive**. `#` starts a comment.

1. Key-value pairs (maps)

maps.yml

```yaml
name: my-app
version: 1.2
enabled: true
```

2. Nested maps: indentation means "child of"

nested.yml

```yaml
server:
  host: localhost
  port: 8080
```

3. Lists: one dash per item

lists.yml

```yaml
branches:
  - main
  - develop

# inline form of the same list
branches: [main, develop]
```

4. List of maps (the shape of every pipeline step list)

list-of-maps.yml

```yaml
steps:
  - name: Build
    run: npm run build
  - name: Test
    run: npm test
```

Each dash starts a new item. Keys that line up under the first key (`name`, `run`) belong to the same item.

5. Data types

types.yml

```yaml
text: hello
quoted: "hello: world"   # quotes needed when text contains : or #
number: 42
float: 3.14
boolean: true
nothing: null
```

6. Multi-line strings

multiline.yml

```yaml
script: |        # keeps line breaks (use this for multi-line commands)
  echo "line 1"
  echo "line 2"
description: >   # folds lines into one line
  This is a long
  sentence.
```

7. Anchors and aliases: reuse a block

anchors.yml

```yaml
defaults: &defaults
  retries: 3
  timeout: 10
job1:
  <<: *defaults
```

Common gotchas

A tab character breaks the file. Configure your editor to insert spaces.

Unquoted `yes`, `no`, `on`, `off` may be read as booleans (YAML 1.1). Quote values like `"3.10"` or they become `3.1`.

A value starting with `${{` or `*`, or containing `: `, often needs quotes.

Wrong indentation does not always give an error: it can silently change the meaning.

2 Schema tables

Every key in Azure DevOps and GitHub Actions, level by level

Mental model: how the two tools map to each other

Concept	Azure DevOps	GitHub Actions
Pipeline file	`azure-pipelines.yml` (any path, registered in UI)	`.github/workflows/*.yml`
Top-level unit	Pipeline	Workflow
Grouping of jobs	`stages` (optional)	None (use `needs` for ordering)
Parallel unit of work	`job`	`job`
Smallest unit	`step` (task or script)	`step` (action or run)
Run a command	`script`, `bash`, `pwsh`	`run`
Reusable step	`task: Name@version`	`uses: owner/action@version`
Build machine	`pool`	`runs-on`
Dependencies	`dependsOn`	`needs`
Conditions	`condition`	`if`
Secrets	Variable groups, Key Vault, secret variables	`secrets.*`
Approvals	Environments with checks	Environments with protection rules
Reuse	`template`	Reusable workflows (`workflow_call`), composite actions

Azure DevOps YAML schema

Pipeline > **Stages** > **Jobs** > **Steps**

You can skip levels. The shortest valid pipeline has only `steps`; the tool fills in the stage and job for you.

Pipeline (root) level

Key	Purpose
`name`	Custom run number format (e.g. 1.0$(Rev:.r))
`trigger`	CI trigger: branches, tags, paths (include / exclude, batch)
`pr`	Pull request validation trigger
`schedules`	Cron-based triggers
`resources`	External dependencies: repositories, containers, pipelines, packages
`variables`	Variables, variable groups, templates
`parameters`	Runtime inputs (typed: string, boolean, number, object, step, ...)
`pool`	Default agent pool (vmImage, name, demands)
`stages / jobs / steps`	Structure of the work (you can skip levels)
`extends`	Force a pipeline to extend a template (governance)
`lockBehavior`	Sequential or runLatest for resource locking
`appendCommitMessageToRunName`	Include the commit message in the run name

Stage level

Key	Purpose
`stage`	Stage identifier
`displayName`	Friendly name in the UI
`dependsOn`	Stages that must finish first
`condition`	When to run, e.g. succeeded() or eq(variables['Build.SourceBranch'], 'refs/heads/main')
`variables`	Stage-scoped variables
`pool`	Agent pool override
`jobs`	List of jobs
`lockBehavior / templateContext`	Locking and template metadata

Job level

Key	Purpose
`job`	Job identifier (regular job)
`deployment`	Identifier for a deployment job (alternative to job)
`displayName`	Friendly name
`dependsOn, condition`	Ordering and conditional run
`pool`	Agent selection
`strategy`	matrix / parallel for regular jobs; runOnce / rolling / canary for deployment jobs
`environment`	Target environment (deployment jobs)
`variables`	Job-scoped variables
`timeoutInMinutes, cancelTimeoutInMinutes`	Time limits
`continueOnError`	Do not fail the pipeline if the job fails
`container, services`	Run the job in a container, with sidecar containers
`workspace`	Clean options for the agent workspace
`steps`	List of steps

Deployment strategy lifecycle hooks: `preDeploy`, `deploy`, `routeTraffic`, `postRouteTraffic`, `on: success / failure`.

Step level

Key	Purpose
`script`	Run a command with the default shell (cmd on Windows, bash on Linux/macOS)
`bash / pwsh / powershell`	Run with a specific shell
`task`	Run a built-in or marketplace task (Name@version), with inputs
`checkout`	Check out a repo (self, or a repo from resources)
`download / publish`	Download or publish pipeline artifacts
`template`	Insert a reusable step template
`displayName`	Name in the UI
`name`	Step reference name (for output variables)
`condition`	Conditional execution
`continueOnError`	Keep going on failure
`enabled`	Disable a step without deleting it
`env`	Environment variables (needed to pass secret variables)
`workingDirectory`	Folder to run in
`timeoutInMinutes`	Step timeout
`retryCountOnTaskFailure`	Automatic retry
`failOnStderr`	Fail if anything is written to stderr
`target`	Run on the host or in a container

Variable syntax (unique to Azure DevOps)

Syntax	Evaluated	Example
`$(var)`	Macro: at runtime, just before a task runs	`$(Build.BuildId)`
`${{ variables.var }}`	Template expression: at compile time	`${{ parameters.env }}`
`$[ variables.var ]`	Runtime expression: for conditions and variable assignment	`$[ eq(variables.x, 'y') ]`

GitHub Actions YAML schema

Workflow > **Jobs** > **Steps**

Workflow (root) level

Key	Purpose
`name`	Workflow name in the Actions tab
`run-name`	Dynamic name for each run
`on`	Events that trigger the workflow
`permissions`	Scopes of the GITHUB_TOKEN (e.g. contents: read)
`env`	Environment variables for the whole workflow
`defaults`	Default shell / working-directory for run steps
`concurrency`	Limit to one running instance per group (group, cancel-in-progress)
`jobs`	The jobs (required)

on: triggers

Event	Notes
`push`	Filters: branches, branches-ignore, tags, paths, paths-ignore
`pull_request`	Same filters plus types (opened, synchronize, reopened, ...)
`workflow_dispatch`	Manual run; supports typed inputs
`schedule`	cron: '0 2 * * *' (UTC)
`workflow_call`	Makes the workflow reusable; defines inputs, secrets, outputs
`workflow_run`	Triggered when another workflow completes
`release, issues, create, delete, ...`	Many other repository events

Job level: jobs.<job_id>

Key	Purpose
`name`	Display name
`runs-on`	Runner (ubuntu-latest, windows-latest, macos-latest, self-hosted labels)
`needs`	Jobs that must complete first
`if`	Conditional execution
`permissions`	Job-level token permissions
`environment`	Deployment environment (approvals, secrets, URL)
`concurrency`	Job-level concurrency group
`outputs`	Values passed to downstream jobs (read via needs.<job>.outputs.x)
`env, defaults`	Job-scoped environment and defaults
`strategy`	matrix, fail-fast, max-parallel
`timeout-minutes`	Time limit (default 360)
`continue-on-error`	Do not fail the workflow if the job fails
`container, services`	Run in a container, with sidecar services (e.g. Postgres)
`steps`	List of steps
`uses / with / secrets`	Call a reusable workflow instead of defining steps

Step level: jobs.<job_id>.steps[*]

Key	Purpose
`name`	Display name
`id`	Reference for outputs (steps.<id>.outputs.x)
`uses`	Run an action (owner/repo@version)
`with`	Inputs to an action
`run`	Run a shell command
`shell`	bash, pwsh, python, cmd, etc.
`env`	Step-scoped environment variables
`if`	Conditional execution
`working-directory`	Folder to run in
`continue-on-error`	Keep going on failure
`timeout-minutes`	Step timeout

Expressions and contexts

Expressions use `${{ ... }}` and are evaluated by GitHub before the step runs.

Context	Example
`github`	`${{ github.sha }} ${{ github.ref }} ${{ github.actor }}`
`env`	`${{ env.NODE_VERSION }}`
`vars`	`${{ vars.MY_CONFIG }} (non-secret configuration variables)`
`secrets`	`${{ secrets.API_KEY }}`
`matrix`	`${{ matrix.node }}`
`needs`	`${{ needs.build.outputs.version }}`
`steps`	`${{ steps.x.outputs.y }}`
`runner, job, inputs`	`Runner info, job status, workflow inputs`

3 Sample pipeline: Node.js

Build, test, publish, then deploy to a dev environment

Both pipelines do the same thing: build and test on every push to `main`, then deploy to a `dev` environment. Compare them side by side to see how the same idea is expressed in each tool.

Azure DevOps: azure-pipelines.yml

azure-pipelines.yml

```yaml
trigger:
  branches:
    include:
      - main

pool:
  vmImage: ubuntu-latest

variables:
  nodeVersion: '20.x'

stages:
  - stage: Build
    jobs:
      - job: BuildAndTest
        steps:
          - task: NodeTool@0
            displayName: Install Node.js
            inputs:
              versionSpec: $(nodeVersion)

          - script: npm ci
            displayName: Install dependencies

          - script: npm test
            displayName: Run tests

          - script: npm run build
            displayName: Build app

          - publish: dist
            artifact: app

  - stage: Deploy
    dependsOn: Build
    condition: succeeded()
    jobs:
      - deployment: DeployToDev
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - script: echo "Deploying artifact 'app' to dev..."
                  displayName: Deploy
```

Line(s)	Why it is there
`trigger: branches: include: [main]`	Run automatically whenever someone pushes to main. Without trigger, ADO defaults to every branch.
`pool: vmImage: ubuntu-latest`	Chooses a Microsoft-hosted Linux agent to run the jobs.
`variables: nodeVersion`	Defines a value once and reuses it as $(nodeVersion). Change it in one place.

Line(s)	Why it is there
`stages: / - stage: Build`	A stage is a major phase. Splitting Build and Deploy lets you add approvals, order, and conditions between them.
`jobs: / - job: BuildAndTest`	A job runs on one agent. All its steps share the same machine and workspace.
`steps:`	The ordered actions inside the job.
`task: NodeTool@0`	Built-in task that installs a specific Node.js version. @0 pins the task major version.
`inputs: versionSpec`	Parameters passed to the task.
`displayName`	A readable label in the pipeline UI and logs.
`script: npm ci`	Runs a shell command. npm ci is a clean, reproducible install from package-lock.json: right for CI.
`script: npm test`	Runs tests. A non-zero exit code fails the step and the pipeline: this is how CI blocks bad code.
`script: npm run build`	Produces the deployable output in dist.
`publish: dist / artifact: app`	Uploads dist as a pipeline artifact named app so a later stage (on another agent) can use it.
`dependsOn: Build`	Deploy waits for Build to finish.
`condition: succeeded()`	Deploy only if everything before succeeded (the default, shown for clarity).
`deployment: DeployToDev`	Special job type for deployments: tracks history per environment and supports strategies.
`environment: dev`	Targets an ADO Environment where you can add manual approvals and checks.
`strategy: runOnce: deploy: steps:`	Runs the deploy steps once. Alternatives: rolling, canary. Deployment jobs automatically download published artifacts.

GitHub Actions: .github/workflows/ci.yml

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

env:
  NODE_VERSION: '20'

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm test

      - name: Build app
        run: npm run build

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: app
          path: dist

  deploy:
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v4
        with:
          name: app

      - name: Deploy
        run: echo "Deploying artifact 'app' to dev..."
```

Line(s)	Why it is there
`name: CI`	The label shown in the repo's Actions tab.
`on: push: branches: [main]`	Run on pushes to main.
`on: pull_request`	Also run on PRs so problems are caught before merging.
`env: NODE_VERSION`	Workflow-wide variable, read with ${{ env.NODE_VERSION }}.
`jobs: / build:`	Defines a job. The key (build) is its ID, used by needs.
`runs-on: ubuntu-latest`	Selects a GitHub-hosted Linux runner.

Line(s)	Why it is there
`uses: actions/checkout@v4`	Required in GitHub Actions: the runner starts empty, so this downloads your repo code. (ADO does this automatically.)
`uses: actions/setup-node@v4`	Official action that installs Node.js. @v4 pins the action version.
`with: node-version`	Inputs passed to the action.
`cache: npm`	Caches npm downloads between runs to speed up builds.
`name: on steps`	Readable labels in the logs.
`run: npm ci / test / build`	Shell commands: clean install, run tests (failure turns the run red), produce dist.
`actions/upload-artifact@v4`	Saves dist as an artifact named app so another job (on a fresh runner) can fetch it.
`deploy: / needs: build`	Deploy runs only after build succeeds. Without needs, jobs run in parallel.
`if: github.ref == 'refs/heads/main'`	Skip deployment on pull requests or other branches.
`environment: dev`	Links to a GitHub Environment where you can require reviewers and scope secrets.
`actions/download-artifact@v4`	Pulls the app artifact into the deploy job. Not automatic as in ADO.

4 Azure build and deploy scenarios

.NET, Java, ACR to AKS, ACR to ACA

Scenario	Packaging	Deploy target	Typical use
.NET	zip / publish folder	Azure App Service	Simplest PaaS hosting
Java	.jar	Azure App Service	Same pattern, different build tool
ACR + AKS	Docker image	Kubernetes cluster	Full control, many microservices
ACR + ACA	Docker image	Azure Container Apps	Containers without managing Kubernetes

Prerequisites (one-time setup, not in YAML)
The Azure resources already exist: App Service, ACR, AKS or Container App.
Azure DevOps needs service connections (Project Settings, Service connections) to Azure, ACR and AKS.
GitHub Actions needs an Azure login using OIDC (an app registration with a federated credential). Add three repo secrets: `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`. This avoids storing passwords.

4.1 .NET to Azure App Service

Azure DevOps

azure-pipelines.yml

```yaml
trigger:
  - main

pool:
  vmImage: ubuntu-latest

variables:
  buildConfiguration: Release

stages:
  - stage: Build
    jobs:
      - job: Build
        steps:
          - task: UseDotNet@2
            inputs:
              packageType: sdk
              version: 10.x

          - script: dotnet restore
            displayName: Restore

          - script: dotnet build --configuration $(buildConfiguration) --no-restore
            displayName: Build

          - script: dotnet test --configuration $(buildConfiguration) --no-build
            displayName: Test

          - task: DotNetCoreCLI@2
            displayName: Publish
            inputs:
              command: publish
              publishWebProjects: true
              arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)'
              zipAfterPublish: true

          - publish: $(Build.ArtifactStagingDirectory)
            artifact: drop

  - stage: Deploy
    dependsOn: Build
    jobs:
      - deployment: Deploy
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
                  inputs:
                    azureSubscription: 'my-azure-connection'
                    appType: webAppLinux
                    appName: 'my-dotnet-app'
                    package: '$(Pipeline.Workspace)/drop/**/*.zip'
```

Line(s)	What and why
`trigger: - main`	Short form of the branch trigger: run on every push to main.
`pool: vmImage`	Microsoft-hosted Ubuntu agent.

Line(s)	What and why
`variables: buildConfiguration`	Defined once, reused as $(buildConfiguration). Release produces optimized output.
`UseDotNet@2 + version: 10.x`	Installs the .NET SDK your app targets so builds do not depend on the agent's preinstalled version.
`dotnet restore`	Downloads NuGet packages.
`dotnet build ... --no-restore`	Compiles. --no-restore skips a second restore.
`dotnet test ... --no-build`	Runs unit tests on the compiled output. A failure stops the pipeline.
`DotNetCoreCLI@2 command: publish`	Produces the deployable output (compiled app plus dependencies).
`publishWebProjects: true`	Auto-finds web projects instead of naming the .csproj.
`--output $(Build.ArtifactStagingDirectory)`	Puts the output in the standard staging folder.
`zipAfterPublish: true`	Creates a single zip, which is what App Service deploys.
`publish: ... artifact: drop`	Uploads the zip so the Deploy stage (different agent) can get it.
`dependsOn: Build`	Deploy waits for Build.
`deployment + environment: dev`	Deployment job tied to an environment (history, approvals, checks).
`runOnce: deploy: steps:`	Deploys one time. The drop artifact is downloaded automatically to $(Pipeline.Workspace)/drop.
`AzureWebApp@1`	Built-in task that deploys a package to App Service.
`azureSubscription`	Name of your Azure service connection (the credentials).
`appType: webAppLinux`	Target is a Linux web app. Use webApp for Windows.
`appName`	The App Service name.
`package: ...**/*.zip`	Glob pattern that finds the zip.

GitHub Actions

.github/workflows/dotnet.yml

```yaml
name: dotnet-ci-cd

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.x

      - run: dotnet restore
      - run: dotnet build --configuration Release --no-restore
      - run: dotnet test --configuration Release --no-build
      - run: dotnet publish -c Release -o ./publish

      - uses: actions/upload-artifact@v4
        with:
          name: app
          path: ./publish

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: app
          path: ./publish

      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - uses: azure/webapps-deploy@v3
        with:
          app-name: my-dotnet-app
          package: ./publish
```

Line(s)	What and why
`on: push: branches: [main]`	Trigger on pushes to main.
`permissions: id-token: write`	Lets the job request an OIDC token: required for passwordless Azure login.
`permissions: contents: read`	Least privilege: only read the repo.
`actions/checkout@v4`	Pulls your code onto the empty runner.
`actions/setup-dotnet@v4`	Installs the .NET SDK.
`dotnet restore / build / test`	Same meaning as in ADO. Each is a run: step.
`dotnet publish -c Release -o ./publish`	Produces the deployable folder. App Service can deploy a folder directly, so no zip needed.

Line(s)	What and why
`upload / download-artifact`	Pass the output between jobs (different machines).
`needs: build`	Deploy runs only after build succeeds.
`environment: dev`	Enables approvals and environment-scoped secrets.
`azure/login@v2`	Logs in to Azure using the OIDC federated credential: no secret password.
`azure/webapps-deploy@v3`	Deploys the package to the named App Service.

4.2 Java (Maven) to Azure App Service

Azure DevOps

azure-pipelines.yml

```yaml
trigger:
  - main

pool:
  vmImage: ubuntu-latest

stages:
  - stage: Build
    jobs:
      - job: Build
        steps:
          - task: Maven@4
            displayName: Build and test
            inputs:
              mavenPomFile: pom.xml
              goals: package
              options: '-B'
              jdkVersionOption: '1.21'
              publishJUnitResults: true
              testResultsFiles: '**/surefire-reports/TEST-*.xml'

          - script: cp target/*.jar $(Build.ArtifactStagingDirectory)/app.jar
            displayName: Stage jar

          - publish: $(Build.ArtifactStagingDirectory)/app.jar
            artifact: drop

  - stage: Deploy
    dependsOn: Build
    jobs:
      - deployment: Deploy
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebApp@1
                  inputs:
                    azureSubscription: 'my-azure-connection'
                    appType: webAppLinux
                    appName: 'my-java-app'
                    runtimeStack: 'JAVA|21-java21'
                    package: '$(Pipeline.Workspace)/drop/app.jar'
```

Line(s)	What and why
`Maven@4`	Built-in task that runs Maven.
`mavenPomFile: pom.xml`	Which project file to build.
`goals: package`	Compiles, runs tests, and creates the .jar in target/.
`options: '-B'`	Batch mode: cleaner logs for CI (no interactive output).
`jdkVersionOption: '1.21'`	Uses Java 21 (ADO writes versions as 1.x).
`publishJUnitResults + testResultsFiles`	Shows test results in the pipeline's Tests tab.
`cp target/*.jar ... app.jar`	Copies the jar to staging with a fixed name. Assumes one runnable jar (e.g. Spring Boot fat jar).
`publish: ... artifact: drop`	Uploads the jar for the Deploy stage.

Line(s)	What and why
`runtimeStack: 'JAVA|21-java21'`	Tells App Service which Java runtime to use (Java 21, Java SE).
`package: .../app.jar`	The jar to deploy.
`Remaining deploy lines`	Same as in the .NET example.

GitHub Actions

`.github/workflows/java.yml`

```yaml
name: java-ci-cd

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - run: mvn -B package

      - run: cp target/*.jar app.jar

      - uses: actions/upload-artifact@v4
        with:
          name: app
          path: app.jar

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: app

      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - uses: azure/webapps-deploy@v3
        with:
          app-name: my-java-app
          package: app.jar
```

Line(s)	What and why
`actions/setup-java@v4`	Installs a JDK.
`distribution: temurin`	Which JDK vendor (Eclipse Temurin is a common free choice). Required in this action.

Line(s)	What and why
`java-version: '21'`	Quoted so YAML treats it as a string.
`cache: maven`	Caches ~/.m2 so dependencies are not redownloaded every run.
`mvn -B package`	Compiles, tests, and packages the jar. A failing test fails the job.
`cp target/*.jar app.jar`	Gives the jar a predictable name.
`Artifact, login, deploy steps`	Same logic as the .NET example. package: app.jar points at the jar.

4.3 Docker image to ACR to AKS

Both pipelines need a `Dockerfile` in the repo and a Kubernetes manifest. This minimal manifest is used by both. The deploy step replaces the `image:` value with the tag you just built.

k8s/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myacr.azurecr.io/myapp
          ports:
            - containerPort: 8080
```

Azure DevOps

azure-pipelines.yml

```yaml
trigger:
  - main

pool:
  vmImage: ubuntu-latest

variables:
  acrName: myacr.azurecr.io
  imageRepo: myapp

stages:
  - stage: Build
    jobs:
      - job: BuildPush
        steps:
          - task: Docker@2
            displayName: Build and push image
            inputs:
              containerRegistry: 'my-acr-connection'
              repository: $(imageRepo)
              command: buildAndPush
              Dockerfile: Dockerfile
              tags: |
                $(Build.BuildId)

  - stage: Deploy
    dependsOn: Build
    jobs:
      - deployment: DeployAKS
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - checkout: self

                - task: KubernetesManifest@1
                  displayName: Deploy to AKS
                  inputs:
                    action: deploy
                    connectionType: kubernetesServiceConnection
                    kubernetesServiceConnection: 'my-aks-connection'
                    namespace: default
                    manifests: k8s/deployment.yaml
                    containers: '$(acrName)/$(imageRepo):$(Build.BuildId)'
```

Line(s)	What and why
`variables: acrName, imageRepo`	Registry address and image name defined once.
`Docker@2`	Built-in task for Docker build / login / push.
`containerRegistry`	Service connection for ACR (provides the login).
`repository`	Image name inside the registry.
`command: buildAndPush`	Builds from the Dockerfile, then pushes to ACR in one step.
`tags: $(Build.BuildId)`	Unique tag per build. Avoid latest in deployments: unique tags are traceable and let you roll back.
`deployment + environment`	Deployment job with approvals support.
`checkout: self`	Deployment jobs do not check out code by default, and this job needs the k8s/ manifest file.

Line(s)	What and why
`KubernetesManifest@1 action: deploy`	Applies manifests to the cluster (like kubectl apply), with rollout status checking.
`kubernetesServiceConnection`	Connection that holds access to the AKS cluster.
`namespace`	Target Kubernetes namespace.
`manifests`	The YAML file(s) to apply.
`containers: ...:$(Build.BuildId)`	Overrides the image in the manifest with the freshly built tag.

GitHub Actions

.github/workflows/aks.yml

```yaml
name: aks-ci-cd

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

env:
  ACR_NAME: myacr
  IMAGE: myapp

jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - run: az acr login --name $ACR_NAME

      - run: |
          docker build -t $ACR_NAME.azurecr.io/$IMAGE:${{ github.sha }} .
          docker push $ACR_NAME.azurecr.io/$IMAGE:${{ github.sha }}

  deploy:
    needs: build-push
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: actions/checkout@v4

      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - uses: azure/aks-set-context@v4
        with:
          resource-group: my-rg
          cluster-name: my-aks

      - uses: azure/k8s-deploy@v5
        with:
          namespace: default
          manifests: k8s/deployment.yaml
          images: myacr.azurecr.io/myapp:${{ github.sha }}
```

Line(s)	What and why
`env: ACR_NAME, IMAGE`	Workflow-wide values: $ACR_NAME in shell, ${{ env.X }} in expressions.
`azure/login@v2`	OIDC login to Azure. Needed in each job because jobs run on separate machines.
`az acr login --name $ACR_NAME`	Authenticates Docker to your registry using the Azure session.
`run: | (multi-line)`	Runs two commands in order: build, then push.

Line(s)	What and why
`:${{ github.sha }}`	Tags the image with the commit SHA: unique and traceable back to the code.
`azure/aks-set-context@v4`	Fetches cluster credentials so later steps can talk to AKS.
`azure/k8s-deploy@v5`	Applies manifests, substitutes the image, and waits for the rollout.
`manifests / images`	The file to apply and the image tag to inject.
`actions/checkout@v4 in deploy`	Needed again to get k8s/deployment.yaml onto this runner.

4.4 Docker image to ACR to Azure Container Apps

ACA has no Kubernetes manifests: you just point the Container App at a new image. That is why it is simpler than AKS. Assume the Container App `my-aca` already exists (create it once via portal, CLI or Bicep).

Azure DevOps

azure-pipelines.yml

```yaml
trigger:
  - main

pool:
  vmImage: ubuntu-latest

variables:
  acrName: myacr.azurecr.io
  imageRepo: myapp

stages:
  - stage: Build
    jobs:
      - job: BuildPush
        steps:
          - task: Docker@2
            inputs:
              containerRegistry: 'my-acr-connection'
              repository: $(imageRepo)
              command: buildAndPush
              Dockerfile: Dockerfile
              tags: |
                $(Build.BuildId)

  - stage: Deploy
    dependsOn: Build
    jobs:
      - deployment: DeployACA
        environment: dev
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureCLI@2
                  displayName: Update Container App
                  inputs:
                    azureSubscription: 'my-azure-connection'
                    scriptType: bash
                    scriptLocation: inlineScript
                    inlineScript: |
                      az containerapp update \
                        --name my-aca \
                        --resource-group my-rg \
                        --image $(acrName)/$(imageRepo):$(Build.BuildId)
```

Line(s)	What and why
`Build stage`	Identical to the AKS version: build the image and push it to ACR with a unique tag.
`AzureCLI@2`	Runs Azure CLI commands, already logged in via the service connection.
`scriptType: bash / inlineScript`	The script is written directly in the YAML.
`az containerapp update`	Changes the existing Container App.
`--name, --resource-group`	Identify which Container App to update.
`--image ...`	The new image tag. ACA creates a new revision and shifts traffic to it.
`\ at end of line`	Continues the command on the next line (readability only).

GitHub Actions

.github/workflows/aca.yml

```yaml
name: aca-ci-cd

on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

env:
  ACR_NAME: myacr
  IMAGE: myapp

jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - run: az acr login --name $ACR_NAME

      - run: |
          docker build -t $ACR_NAME.azurecr.io/$IMAGE:${{ github.sha }} .
          docker push $ACR_NAME.azurecr.io/$IMAGE:${{ github.sha }}

  deploy:
    needs: build-push
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

      - uses: azure/cli@v2
        with:
          inlineScript: |
            az containerapp update \
              --name my-aca \
              --resource-group my-rg \
              --image ${{ env.ACR_NAME }}.azurecr.io/${{ env.IMAGE }}:${{ github.sha }}
```

Line(s)	What and why
`build-push job`	Same as the AKS version: log in, authenticate Docker to ACR, build, push.
`No checkout in deploy`	Unlike AKS, there is no manifest file to read. The only input is the image tag.
`azure/cli@v2 + inlineScript`	Runs Azure CLI commands in an authenticated session.
`az containerapp update --image`	Creates a new revision running the new image.

5 Cheat sheet and next steps

Key differences and what to learn next

Key differences to remember

Topic	Azure DevOps	GitHub Actions
Checkout	Automatic in regular jobs	Needs `actions/checkout`
Artifacts	Deployment jobs download them automatically	You download explicitly with `download-artifact`
Stages	Built-in `stages` layer	Use `needs` between jobs
Reusable steps	`task:` (marketplace tasks)	`uses:` (actions)
Variables	Three syntaxes: `$()`, `${{ }}`, `$[ ]`	Mostly `${{ }}`
Azure authentication	Service connections	OIDC login with `azure/login`
Approvals	Environment checks	Environment protection rules

AKS vs ACA

	AKS	ACA
Deploy mechanism	Kubernetes manifests (kubectl apply equivalent)	az containerapp update --image
Files in repo	Dockerfile + k8s/*.yaml	Dockerfile only
Pipeline complexity	Higher	Lower
Control	Full Kubernetes	Simplified, serverless-style
Scale to zero	Needs add-ons (e.g. KEDA)	Built in

Improvements to learn next

- **Run tests before building the image.** The container pipelines skip tests to stay short; add a test step first.
- **Use `dotnet` / `mvn` inside the Dockerfile** (multi-stage builds) so the image build is self-contained.
- **Add a manual approval** on the dev / prod environments.
- **Use separate environments** (dev, staging, prod) with the same pipeline structure.
- **Add Bicep or Terraform** so App Service, AKS and ACA are created by a pipeline too.
- Add a `matrix` (GitHub) or `strategy: matrix` (ADO) to test several runtime versions.
- Learn secrets, then templates and reusable workflows, then real deployment targets.

Suggested practice path

Step	Exercise	You will learn
1	Create a small repo with a hello-world Node or Python app and run the GitHub Actions Node.js pipeline	Triggers, jobs, steps
2	Break a test on purpose and watch the pipeline fail	How CI blocks bad code
3	Add a matrix to test multiple versions	Parallel runs
4	Add an environment with a required reviewer	Approvals
5	Port the same pipeline to Azure DevOps	Schema differences
6	Deploy a container to ACA, then to AKS	Real delivery to Azure